

3DGS-CD: 3D Gaussian Splatting-Based Change Detection for Physical Object Rearrangement

Ziqi Lu , Jianbo Ye, and John Leonard , *Fellow, IEEE*

Abstract—We present 3DGS-CD, the first 3D Gaussian Splatting (3DGS)-based method for detecting physical object rearrangements in 3D scenes. Our approach estimates 3D object-level changes by comparing two sets of unaligned images taken at different times. Leveraging 3DGS’s novel view rendering and EfficientSAM’s zero-shot segmentation capabilities, we detect 2D object-level changes, which are then associated and fused across views to estimate 3D change masks and object transformations. Our method can *accurately* identify changes in *cluttered environments* using *sparse* (as few as one) post-change images within as little as 18 s. It does not rely on depth input, user instructions, pre-defined object classes, or object models – An object is recognized simply if it has been re-arranged. Our approach is evaluated on both public and self-collected real-world datasets, achieving up to 14% *higher accuracy* and *three orders of magnitude faster performance* compared to the state-of-the-art radiance-field-based change detection method. This significant performance boost enables a broad range of downstream applications, where we highlight three *key use cases*: object reconstruction, robot workspace reset, and 3DGS model update.

Index Terms—Computer vision for automation, mapping.

I. INTRODUCTION

3D CHANGE detection involves identifying changed objects or regions in the environment from two sets of local observations taken at different times or from new observations of a previously modeled scene. It is a critical task in robotics as it can accommodate not only short-term, continuously observed scene dynamics but also long-term scene changes where the transition process is typically unobserved. Capturing such long-term changes is particularly important for robot operations, as environmental changes often occur without being noticed or monitored.

Despite the success of 3D change detection with depth input using scene representations such as TSDF [1], 3D point cloud [2] and neural descriptor fields [3], detecting changes

from multi-view RGB images remains a challenging problem. Traditional methods have relied on hand-crafted techniques like voxelization [4], multi-view stereo [5], and image warping [6] to identify changes in unaligned images and lift to 3D. These approaches are particularly sensitive to occlusions and lighting variations, especially when there are large viewpoint differences between the two sets of images.

The emergence of radiance field models, such as neural radiance fields (NeRF) [7] and 3DGS [8], presents new opportunities to address these challenges. These models provide high-fidelity representations of scene geometry and appearance, with novel-view rendering capabilities that enable the generation of photo-realistic images and dense depths at arbitrary viewpoints. This allows for direct comparison of pre- and post-change images from the same viewpoint.

NeRF-based solutions have been explored to a limited extent [9], [10]. However, these methods are constrained by the computational cost of NeRF’s ray-casting-based rendering. In contrast, 3DGS offers a more efficient alternative, achieving real-time rendering with comparable or even superior quality.

In our work, we leverage 3DGS as scene representation to identify 3D scene changes including object removal, insertion and movement from multi-view images. We exploit the zero-shot segmentation capability of EfficientSAM [11] to compare pre- and post-change images at the same viewpoints, associating and fusing the 2D object changes to obtain accurate 3D object segments and pose changes.

Our method has the following key advantages: (1) It can handle **sparse** post-change image inputs, requiring as few as *a single new image* to detect 3D changes. (2) It requires no depth sensors or monocular depth estimators. (3) It does not rely on pre-defined object classes, models or object detectors. (4) It requires no user instructions such as user-provided click or language prompts.

Our method is evaluated on both public and self-collected real-world datasets, achieving up to **14% higher accuracy** and **three magnitudes faster** performance compared to the state-of-the-art NeRF-based method. This significant performance improvement enables a wide range of real-world applications, including: (1) object removal as a prompt for object reconstruction (Section VI-A); (2) robot workspace reset (Section VI-B); (3) 3DGS model update (Section VI-C).

II. RELATED WORK

A. Change Detection From RGB Images

2D change detection from image pairs has been a long-studied problem. Most existing methods assume minimal viewpoint differences between images (e.g. [12]). While some approaches can handle larger viewpoint shifts (e.g. [13]), they are often

Received 5 November 2024; accepted 5 January 2025. Date of publication 24 January 2025; date of current version 6 February 2025. This article was recommended for publication by Associate Editor Z. Cui and Editor C. Cadena upon evaluation of the reviewers’ comments. This work was supported in part by ONR Neuro-Autonomy MURI under Grant N00014-19-1-2571, in part by ONR DURIP under Grant N00014-23-12164, and in part by the MIT Portugal Program. (Corresponding author: Ziqi Lu.)

Ziqi Lu and John Leonard are with the Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge, MA 02139 USA (e-mail: ziqilu@mit.edu; jleonard@mit.edu).

Jianbo Ye can be reached at USA (e-mail: jianboye.ai@gmail.com).

Our code and data will be made available at <https://github.com/520xyxyzq/3DGS-CD>.

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2025.3533457>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2025.3533457

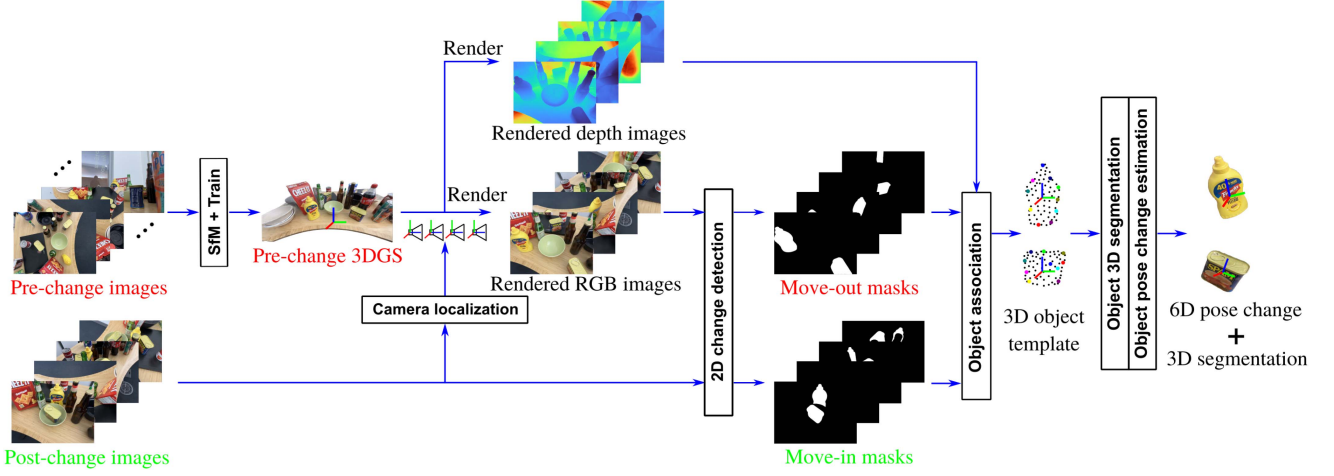


Fig. 1. Our method detects 3D object-level changes from pre- and potentially sparse post-change images of a 3D scene. We first train a 3DGS model on pre-change images (Section IV-A), localize the post-change cameras with respect to this model (Section IV-B), and render RGB-D images at post-change views for 2D change detection using EfficientSAM [11] (Section IV-C). The detected 2D object segments are associated across the post-change views (Section IV-D) to initialize 3D object templates. These templates are used to classify object change types and query EfficientSAM on pre-change views to obtain additional 2D object segments, which are fused to obtain 3D object segments (Section IV-E). For moved objects, we leverage image-template feature correspondences to estimate their 6D pose changes (Section IV-F) and refine the estimates by a render-and-compare approach (Section IV-G).

restricted to the provided views and cannot fully capture physical changes in 3D.

Traditionally, 3D change detection from multi-view RGB images has relied on hand-crafted techniques to identify changes from unaligned images [4], [5], [6]. NeRF-based methods have been explored to directly compare pre- and post-change images from the same viewpoints, but only to a limited extent. NeRF-Update [9] combines NeRF with the Segment Anything Model (SAM) [14] to detect object movements, while C-NeRF [10] models scene changes as direction-consistent radiance differences between two NeRFs. However, these methods are limited by NeRF’s slow rendering speed. In contrast, we leverage 3DGS’s real-time rendering capability for efficient 3D change detection.

B. 3DGS as Scene Representation

3DGS has recently emerged as a key scene representation in robotics but it lacks native support for change representation. While dynamic 3DGS methods (e.g. [15]) can capture short-term dynamics in video sequences, many of them typically rely on continuous spatio-temporal observation of the dynamic process. Our method addresses this gap by detecting changes in 3DGS-represented scenes, offering a more robust solution for capturing 3D scene dynamics.

III. BACKGROUND

A. 3D Gaussian Splatting (3DGS)

3DGS [8] uses a collection of 3D Gaussians $\mathcal{G}$ to explicitly encode the geometry and appearance of a 3D scene. Each 3D Gaussian is defined by its mean (position) $\mu \in \mathbb{R}^3$, covariance matrix $\Sigma \in \mathbb{R}^{3 \times 3}$, opacity $\alpha \in \mathbb{R}$ and color $c \in \mathbb{R}^3$. To render an image from a given view with known camera parameters, the 3D Gaussians are projected (i.e. splatted) onto the camera’s 2D image plane. For each pixel, the subset of 3D Gaussians $\mathcal{G}_p$ that project onto that pixel are gathered and sorted by depth.

The color of a pixel p can be computed from the splatted Gaussians using alpha-compositing as:

$$C(p) = \sum_{i \in \mathcal{G}_p} c_i \alpha_i(p) \prod_{j=1}^{i-1} (1 - \alpha_j(p)) \quad (1)$$

where c_i is the color of the i -th Gaussian and $\alpha_i(p)$ is the opacity of the i -th splatted Gaussian evaluated at pixel p .

The depth for pixel p can be similarly computed by:

$$D(p) = \sum_{i \in \mathcal{G}_p} d_i \alpha_i(p) \prod_{j=1}^{i-1} (1 - \alpha_j(p)) \quad (2)$$

where d_i is the depth of the i -th splatted Gaussian.

During 3DGS training, the Gaussian parameters are optimized by minimizing the error between the rendered and ground truth images over a set of training views. The optimization uses a loss function that combines pixel-wise L1 loss with the structural loss D-SSIM.

B. EfficientSAM

The segment anything model (SAM) [14] is a foundation model for image segmentation from input prompts such as boxes or points. EfficientSAM [11] is a light-weight SAM model enabled by SAM-leveraged masked image pretraining. EfficientSAM adopts an encoder-decoder architecture, i.e. $\mathcal{S} = \mathcal{S}_e \circ \mathcal{S}_d$. The encoder $\mathcal{S}_e$ predicts the embedding $f \in \mathbb{R}^{h \times w \times d}$ for input image $I \in \mathbb{R}^{H \times W \times 3}$, and the decoder $\mathcal{S}_d$ takes the image embedding f and the prompt $\mathcal{P}$ to output the segmentation mask and the prediction confidence.

$$\mathbf{M}, \mathbf{C} = \mathcal{S}(I, \mathcal{P}) = \mathcal{S}_d(\mathcal{S}_e(I), \mathcal{P}) \quad (3)$$

In our work, we use the EfficientSAM image embedding $f = \mathcal{S}_e(I)$ for 2D object-level change detection, and we use box prompts $\mathcal{B}$ to query EfficientSAM to obtain 2D object segments and confidence, i.e. $\mathbf{M}, \mathbf{C} = \mathcal{S}(I, \mathcal{B})$.

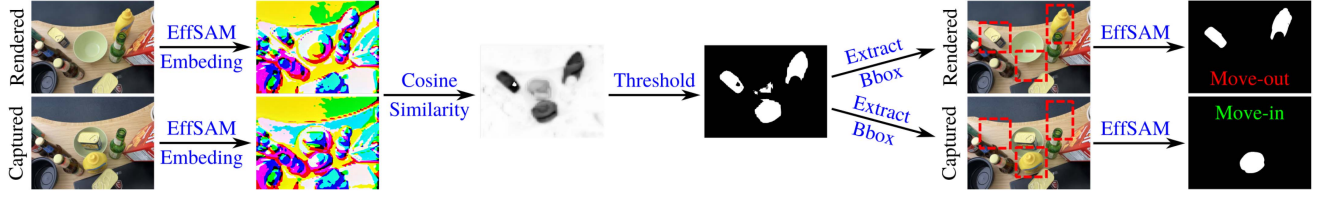


Fig. 2. EfficientSAM embedding for 2D change detection at a post-change view (Section IV-C). While the 2D object masks may be incomplete or missing due to occlusions or prediction failures, associating and fusing masks from multiple views (Section IV-D) help recover more complete object templates.

IV. METHODOLOGY

Our method aims to detect object-level changes in a 3D scene from two sets of RGB images: (1) Pre-change images $\{I_m\}_{m=1}^M$ capturing the initial static state of the scene. (2) Potentially sparse post-change images $\{I'_n\}_{n=1}^N$ observing the changed state of the scene. Our method outputs (1) 3D segmentation $\mathcal{M} : \mathbb{R}^3 \rightarrow \{0, 1\}$ and (2) pose change $\mathcal{T} \in \text{SE}(3)$ for each re-arranged object.

The pipeline of our method is developed to be modular. As shown in Fig. 1, it consists of the following submodules: (1) Pre-change 3DGS training (Section IV-A); (2) post-change camera localization (Section IV-B); (3) 2D change detection on post-change views (Section IV-C); (4) Object association across post-change views (Section IV-D); (5) Object 3D segmentation (Section IV-E); (6) Object pose change estimation (Section IV-F) and refinement (Section IV-G). Beyond that, we also discuss how to project the 3D object segmentations to novel views with occlusion handling (Section IV-H) and provide implementation details in Section IV-I.

A. Pre-Change 3DGS Training

We first train a 3DGS model $\mathcal{G}$ on the pre-change images $\{I_m\}_{m=1}^M$. Following the standard 3DGS training procedure [8], we use a structure-from-motion (SfM) algorithm [16] to estimate pre-change camera parameters and generate a sparse point cloud, initializing 3D Gaussians for subsequent optimization (i.e. training).¹

B. Post-Change Camera Localization

Within the coordinate frame of the pre-change 3DGS, we estimate the camera poses $\{\mathcal{T}_{c_n} \in \text{SE}(3)\}_{n=1}^N$ of the post-change images $\{I'_n\}_{n=1}^N$. Assuming most of the scene observed by the post-change images remains unchanged, we use the standard visual localization procedure [16] to localize the post-change cameras against the SfM point cloud. These initial pose estimates will be further refined in Section IV-G.

C. EfficientSAM Embedding for 2D Change Detection

At the post-change camera views, we use the pre-change 3DGS to render RGB images and compare them with the captured post-change images to detect object-level changes.

We first coarsely detect 2D changes with EfficientSAM [11] image embeddings (Fig. 2). The SAM [14] image embedding, which encodes rich semantic information, has been shown to

enable robust zero-shot change detection [9], [18]. However, probing SAM's latent space can be computationally expensive. We therefore adopt the lightweight EfficientSAM encoder to extract image embeddings on the rendered and captured post-change images, and threshold their cosine similarity maps to obtain the change masks:

$$\mathbf{M}_n^{\text{change}} = \langle \mathcal{S}_e[\mathcal{R}(\mathcal{T}_{c_n}; \mathcal{G})], \mathcal{S}_e(I'_n) \rangle > \text{thresh}. \quad (4)$$

Here, $\mathcal{R}(\mathcal{T}_{c_n}; \mathcal{G})$ denotes the rendered image at view n , the image embeddings are upsampled to the original image resolution before the differencing, and the threshold is dynamically set by Otsu's method [19].

To mitigate the influence of image misalignment and 3DGS floating artifacts on the detection results, we follow [9] to pre-align and blur the rendered and captured images.

From the change masks, we extract contours occupying substantial 2D areas and use their enclosing bounding boxes to query EfficientSAM on the rendered and captured images:

$$\begin{aligned} \mathbf{M}_n, \mathbf{C}_n &= \mathcal{S}[\mathcal{R}(\mathcal{T}_{c_n}), \mathcal{B}(\mathbf{M}_n^{\text{change}})] \\ \mathbf{M}'_n, \mathbf{C}'_n &= \mathcal{S}[I'_n, \mathcal{B}(\mathbf{M}_n^{\text{change}})] \end{aligned} \quad (5)$$

As shown in Fig. 2, the high-confidence 2D segments on the rendered images are used as move-out masks (previous location) for the re-arranged objects, whereas the high-confidence masks on the captured images are identified as object move-in masks (new location):

$$\begin{aligned} \{\mathbf{M}_{n,l}\}_{l=1}^L &= \{\mathbf{M}_n \mid \mathbf{C}_n > 0.95\} \\ \{\mathbf{M}'_{n,l}\}_{l=1}^{L'} &= \{\mathbf{M}'_n \mid \mathbf{C}'_n > 0.95\} \end{aligned} \quad (6)$$

where l indexes the high-confidence masks for view n .

D. Change-Aware Object Association

We associate the 2D object segments across post-change camera views to initialize the re-arranged objects in 3D. As Fig. 1 shows, each initialized object template consists of a dense point cloud and a sparse visual feature point cloud. These templates are built incrementally by aggregating spatially proximate and semantically similar object segments across views.

We first use the pre-change 3DGS to render depths within the object move-out masks and back-project them to 3D to get per-view partial object point clouds:

$$X_{n,l} = \pi^{-1}[\mathcal{R}^d(\mathcal{T}_{c_n}) \cdot \mathbf{M}_{n,l}] \quad (7)$$

where $\pi^{-1}(\cdot)$ represents the back-projection function and $\mathcal{R}^d(\cdot)$ denotes the 3DGS depth rendering function as in (2).

¹For the success of SfM, we typically need the pre-change images to sufficiently cover the pre-change scene. But this requirement can be relaxed with sparse-view 3DGS training techniques, e.g. InstantSplat [17].

We also extract the EfficientSAM embeddings for the object move-out masks and compute their L1-median to obtain per-view per-object representative embedding vectors:

$$f_{n,l} = \text{med} [S_e(\mathcal{R}(\mathcal{T}_{c'_n})) \cdot M_{n,l}] \quad (8)$$

These embedding vectors are only used during object association and will not be included in the object templates.

We further detect sparse visual features (e.g. SuperPoint [20]) within the object move-out masks and back-project them to 3D to form the per-view sparse feature point clouds:

$$F_{n,l} = \pi^{-1} \{ \phi [\mathcal{R}(\mathcal{T}_{c'_n}) \cdot M_{n,l}] \} \quad (9)$$

where ϕ is the feature detection function, and π^{-1} back-projects the 2D feature locations to 3D.

Starting from the object move-out masks on the first view, we iteratively associate the segments across remaining views by matching the partial point clouds. Specifically, we use the Hungarian algorithm to perform nearest-neighbor matching between the current object point cloud and the next segment with the Hausdorff distance. The matched new segments are added to the object templates while the unmatched segments are used to initialize new object templates. We also verify the semantic similarity of the matched segments by comparing their EfficientSAM embeddings. If their cosine similarity is below 40%, a new object template is created instead for the new segment. After the association is done, we obtain the object templates as the union of multi-view point clouds and features:

$$\{O_k = \{X_k, F_k\} = \cup_n \{X_{n,k}, F_{n,k}\}\}_{k=1}^K \quad (10)$$

where k indexes the high-confidence masks across post-change views that associates to the k -th object.

Note that associating object move-out masks only initializes objects at their pre-change locations. We also need to associate the object templates with the object move-in masks to identify the object re-arrangement. We similarly extract the multi-view EfficientSAM embeddings and visual features within the object move-in masks:

$$\begin{aligned} f'_{n,l} &= \text{med} [S_e(I'_n) \cdot M'_{n,l}] \\ F'_{n,l} &= \phi (I'_n \cdot M'_{n,l}) \end{aligned} \quad (11)$$

We match the 2D object features $F'_{n,l}$ on post-change images with 3D template features F_k to establish 2D-3D correspondences. Based on the matching results, we can categorize the object-level changes as:

- **Moved object:** object template with sufficient inlier 2D-3D correspondences
- **Removed object:** object template without sufficient inlier 2D-3D correspondences
- **Inserted object:** post-change object segments without sufficient inlier matches to any object template

The inlier matches are identified with the RANSAC-PnP algorithm (Section IV-F).

For “inserted” post-change object segments, we adopt the same object association procedure as above to initialize their object templates. Since depths are unavailable for post-change views, we match the segments using cosine similarity of the embedding vectors $f'_{n,l}$.

E. Object 3D Segmentation

We perform multi-view mask fusion to establish the 3D segmentation mask $\mathcal{M}_k$ for each object template O_k .

Since the move-out masks $M_{n,l}$ on potentially sparse post-change views may not be sufficient for accurate fusion, we project the point clouds of *moved* and *removed* objects² onto pre-change images and query EfficientSAM with their 2D bounding boxes for additional object move-out masks. To ensure robustness against occlusion, we only fuse non- or slightly-occluded object masks, filtering out those that enclose less than 80% of the projected 2D object points.

For multi-view mask fusion, we initialize a 3D binary voxel grid $\mathcal{M}_k \in \{0, 1\}^{N_x \times N_y \times N_z}$ around each object template. A given voxel x is considered inside the object if it projects into the majority of the object’s 2D masks:

$$\mathcal{M}_k(x) = \frac{1}{N_k} \sum_{n_k=1}^{N_k} \mathbb{1}_{\{M_{n_k,k}[\pi(x)] = 1\}} > 0.95 \quad (12)$$

where π is the projection function and $\mathbb{1}$ the indicator function. n_k indexes all pre- and post-change images that has high-confidence low-occlusion masks for the k -th object. At runtime, the object voxel grids can be queried quickly with arbitrary 3D positions using trilinear interpolation, and a point is considered inside the object if its occupancy probability is above 0.5.

F. Object Pose Change Estimation

For *moved objects*, we use the 2D-3D correspondences from Section IV-D, and apply the standard RANSAC-PnP algorithm to coarsely estimate the object pose changes. The initial estimates are further optimized in Section IV-G.

G. Global Pose Refinement

We refine the initial camera and object pose change estimates with an analysis-by-synthesis approach.

With the initial estimates, we transform the 3D Gaussians in the pre-change 3DGS model, moving those within object 3D masks according to the estimated pose changes. We use the transformed 3D Gaussians, denoted $\mathcal{G}^T$, to render images at the post-change views, which are compared to the captured post-change images. We freeze the pre-change Gaussian parameters and minimize the photometric errors between the rendered and captured images to refine the initial pose estimates:

$$\mathcal{T}_{c'_n}^*, \mathcal{T}_k^* = \arg \min_{\mathcal{T}_{c'_n}, \mathcal{T}_k} \sum_{n=1}^N \mathcal{L} \{ \mathcal{R}(\mathcal{T}_{c'_n}; \mathcal{G}^T) \bar{M}_n, I'_n \bar{M}_n \} \quad (13)$$

Here, $\bar{M}_n$ is the inverted object move-out masks on view n , used to exclude the influence of previously un-observed regions in pre-change 3DGS revealed by the object rearrangement. $\mathcal{L}$ represents the 3DGS training loss [8]. The above optimization is solved using standard gradient descent.

²This step is skipped for *inserted objects* since they don’t have move-out masks. We therefore require more post-change images to more accurately segment inserted objects.



Fig. 3. Qualitative results: Estimated 3D object masks projected on novel views. We show the projected object move-out (previous location) and move-in (new location) masks on the pre-change 3DGS render (left) and the post-change capture (right) from the same novel viewpoint for each scene. Please check out the supplementary video for mask projections on more views.

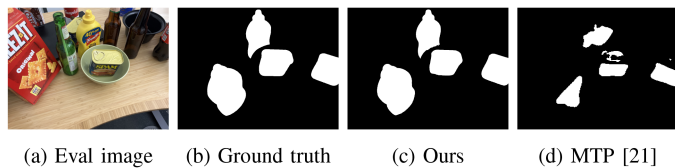


Fig. 4. Qualitative comparison of our method against SOTA 2D change detection method MTP [21] on *Desk*.

H. Occlusion-Aware Mask Projection

Given a pre-change or post-change *evaluation* image, how do we project the 3D object segmentations to its camera view in an accurate and *occlusion-aware* manner?

We first estimate and refine the evaluation-view camera poses as outlined in Sections IV-B and IV-G. We transform the occupied object voxels if the evaluation image is a post-change image and project the voxels to the evaluation images to get initial masks. Depths are then rendered within the initial masks using either the pre-change 3DGS $\mathcal{G}$ (for pre-change images) or the transformed 3DGS $\mathcal{G}^T$ (for post-change images), and back-projected to 3D. Occlusion is checked pixel-wise by determining if the back-projected depths lie inside the 3D segmentations. The occluded pixels are reset to 0 to obtain the final evaluation-view masks.

I. Implementation Details

Our method builds on top of NeRFStudio's implementation of 3DGS: gsplat [22]. We use HLoc [16] for SfM (Section IV-A) and camera localization (Section IV-B). SuperPoint+LightGlue [20], [23] are used for feature detection and matching (Section IV-D).

V. EXPERIMENTS

A. Datasets

To our knowledge, the only public dataset tailored for radiance-field-based change detection is the NeRF-CD dataset [10], which focuses on simple types of changes, such as removal and insertion of objects or object components.

To evaluate our method in more complex scenarios, we collect a challenging real-world dataset, 3DGS-CD, where objects

experience various types of re-arrangements including object moving, swapping and insertion in cluttered environments. The dataset contains 5 scenes: *Mustard*, *Desk*, *Swap*, *Bench*, and *Sill*. Each scene contains about 150 to 200 pre-change images and 8 post-change images, except for *Sill*, which has 60 post-change images to aid segmentation of the inserted object. Half of the post-change images are used for change detection, and the other half for evaluation.

For a fair comparison against the C-NeRF [10] baseline, we also evaluate our method on the first test scene *Cats* of the NeRF-CD [10] dataset, which focuses on object-level changes (object removal). Instead of using all post-change images from the scene, we sampled 4 for change detection. The original pre-change evaluation images provided in the dataset are used for evaluation.

B. Metrics

We evaluate our method on 2D change masks (move-in + move-out) at evaluation views, comparing them against ground truth using Precision(%), Recall(%), F1(%), and IoU(%), as is standard in 2D change detection.

We also report runtime performance, which includes the time for change detection and mask projection or prediction, but excludes NeRF or 3DGS training times.

C. Baselines

MTP [21]: A SOTA 2D change detection method. It is a foundation model pre-trained on remote sensing datasets for multiple tasks and fine-tuned for 2D change detection.

C-NeRF [10]: A SOTA NeRF-based 3D change detection method. C-NeRF trains two NeRFs on the pre- and post-change images in the same coordinate frame, established via joint SfM. It identifies direction-consistent radiance differences between the two NeRFs in 3D, and renders 2D change masks at novel views.

C-NeRF-Diff [10]: A baseline implemented by C-NeRF authors to compute the photometric differences between the images rendered by the pre- and post-change NeRFs.

D-NeRF-Diff [24]: Another baseline by C-NeRF authors that trains a D-NeRF with the two image sets as two time steps. It

TABLE I
QUANTITATIVE EVALUATION OF OUR METHOD ON 3DGS-CD DATASET

Data	Methods	Precision $\uparrow$	Recall $\uparrow$	F1 $\uparrow$	IoU $\uparrow$	Time
Mustard	Ours	97.68	98.75	98.21	96.48	19s
	MTP	94.88	23.08	37.13	22.80	1s
Desk	Ours	99.17	95.82	97.47	95.06	21s
	MTP	95.69	34.38	50.58	33.85	1s
Swap	Ours	98.84	98.82	98.83	97.69	28s
	MTP	94.19	24.61	39.03	24.25	1s
Bench	Ours	96.51	96.76	96.64	93.49	58s
	MTP	90.19	88.74	89.46	80.93	1s
Sill	Ours	95.00	97.86	96.41	93.07	18s
	MTP	48.25	30.84	37.63	23.17	2s

TABLE II
QUANTITATIVE EVALUATION OF OUR METHOD ON THE TEST SCENE *CATS*
FROM THE NeRF-CD [10] DATASET

Methods	Precision $\uparrow$	Recall $\uparrow$	F1 $\uparrow$	IoU $\uparrow$	Time
3DGS-CD	97.94	95.62	96.76	93.73	22s
C-NeRF	86.70	90.46	88.54	79.47	3.4h
C-NeRF-Diff	58.80	83.50	68.86	52.70	-
D-NeRF-Diff	59.24	60.57	59.68	42.71	-
4DGS-Diff	62.44	90.73	73.97	58.69	-

Runtimes for the *-Diff baselines are omitted as they don't have an explicit change detection step.

computes change masks using the absolute color differences between evaluation-view D-NeRF renders.

4DGS-Diff [15]: Similar to D-NeRF-Diff, we train 4DGS at two time steps and compute color differences between eval-view renders at different times to get the change masks.

On the 3DGS-CD dataset, we use MTP as baseline to predict change masks on rendered and captured images at evaluation views.³ We adopt the *ViT-B+RVSA* model fine-tuned on the CDD dataset [25] as it performs best on our data. On the NeRF-CD dataset, we compare our method with C-NeRF, C-NeRF-Diff, D-NeRF-Diff and 4DGS-Diff.

D. Results

For the 3DGS-CD dataset, we present our quantitative results in Table I and qualitative results in Figs. 3 and 4. Our method demonstrates high accuracy across all test cases, with all metrics consistently exceeding 90%. It only takes around 18 s to 58 s to detect 3D changes in a complex scene.

Our method is more accurate than the MTP baseline by a large margin, despite having a slower runtime due to the inference efficiency of MTP's neural network. While MTP's prediction accuracy could be improved through fine-tuning on in-domain data, our method exhibits strong zero-shot performance without any fine-tuning.

We further report our results on the "Cats" scene from the NeRF-CD dataset in Table II and Fig. 3(f). Our method is up to 14% more accurate and 556 times faster than C-NeRF. This significant speedup is primarily due to C-NeRF's reliance on computationally expensive ray tracing to identify changes and render masks, whereas our method leverages EfficientSAM, real-time 3DGS rendering and fast classic computer vision techniques to build our pipeline.

³C-NeRF is not used as a baseline on 3DGS-CD since it fails to train a post-change NeRF with reasonable quality using the sparse new images.

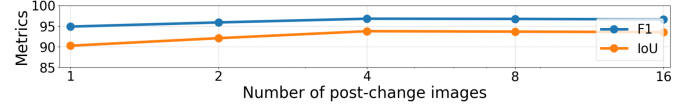


Fig. 5. Ablation study on the number of post-change images on *Cats*. Our method maintains consistent performance across different numbers of post-change images.

E. Ablation Study

1) Number of Post-Change Images: How does our method scale to fewer or more post-change images? We sample and use different numbers of post-change images from the *Cats* scene for change detection, and evaluate our method with the same evaluation images. As Fig. 5 shows, our method maintains stable performance across varying numbers of post-change images, even when using just one image. The slight decline in accuracy with fewer images is mainly due to the incomplete object template built from limited views, which degrades the quality of the box prompt for EfficientSAM on pre-change images (Section IV-E).

VI. APPLICATIONS

Our method significantly advances the accuracy and efficiency of radiance-field-based 3D change detection, enabling a wide range of real-world applications. Below, we show three examples to demonstrate its potential use cases.

A. Object Removal as Prompt for Object Reconstruction

The emergence of NeRF and 3DGS has made the task of 3D object reconstruction more accessible to non-professional users. With just a phone, people can now casually capture RGB images to build an object-only NeRF or 3DGS model in minutes. For complex objects in cluttered environments, however, user prompts are often needed to consistently isolate the same foreground object on the input images.

Common prompts like clicks and language can be ambiguous in challenging scenarios. For instance, click prompts, particularly single clicks, struggle with multi-level granularity where a click may represent an object component or the entire object. Language prompts can also lead to confusion, especially when multiple similar objects are present.

To illustrate this, we use the SOTA prompt-based 3DGS segmentation method SAGA [26] to isolate a target object from its 3DGS-represented scene, as shown in Fig. 6. While SAGA explicitly accounts for multi-level granularity, it still struggles to segment the entire object with a single click prompt (Fig. 6(b) or Fig. 6(c)), even with the best thresholds. We had to provide multiple precise clicks Fig. 6(d) to extract the complete object. However, providing such precise clicks can be difficult on mobile devices. Similarly, using a language prompt generated by GPT-4o also failed to isolate the whole object.

To address this, we propose using **object removal as an unambiguous prompt**, as shown in Fig. 7. After capturing images of the object in its scene, users only have to remove the object, and take a few more images for the object-removed scene. Our method can then consistently identify the target object from the pre-change images, allowing us to train an object-3DGS without ambiguity. Compared to the click or language prompts,

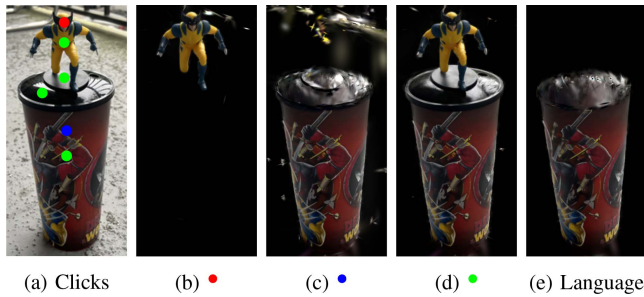


Fig. 6. Click or language prompts can be ambiguous for object extraction: SAGA [26] struggles to segment the entire object with single (a) click prompts on (b) the figurine or (c) the cup body. We had to use (d) multiple precise click prompts to extract the complete object. Using GPT-4o generated language prompt “Coke cup with a wolverine figurine on the lid” also failed to isolate the whole object.

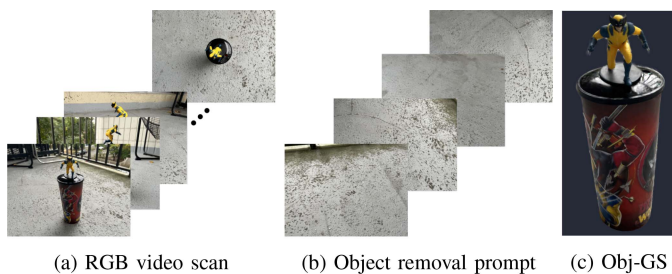


Fig. 7. Object removal as prompt for 3D object reconstruction: Our method only requires a few more images for the object-removed scene to reconstruct the target object without ambiguity.

the object removal prompt is more intuitive and requires minimal user efforts.

B. Robot Workspace Reset

Our method precisely estimates the shapes and pose changes of moved objects, enabling a robotic manipulator to reset the scene to its initial state. This capability is particularly useful in applications such as warehouse or housekeeping robots, where resetting the workspace after operation can be crucial.

The robot workspace reset process can be summarized as follows: (1) RGB video scan to capture the initial state of the robot workspace; (2) Move objects in the workspace; (3) Capture a few more RGB images for the changed state of the workspace; (4) Use our method to estimate the moved objects’ shapes and pose changes in under 30 s; (5) Robot pick and place to reset the state of the scene.

Key advantages of our robot workspace reset method over existing solutions include: (1) No depth sensor or estimator is required; (2) No need for object class, model or object detectors; (3) No need for potentially ambiguous language prompts; (4) *Precise* reset of the object position and orientation.

Fig. 8 shows a simulated experiment we conducted in PyBullet [27], where a robot arm resets the pose of a mug on a table. In this experiment, we used forward kinematics, rather than structure-from-motion (SfM) or visual localization, to estimate the pre- and post-change poses of the robot’s RGB camera. Without the SfM point cloud, the pre-change 3D Gaussians are randomly initialized before optimization. Once the middle mug’s pose change and shape are computed, we determine the pick-and-place poses for the robot gripper and use inverse

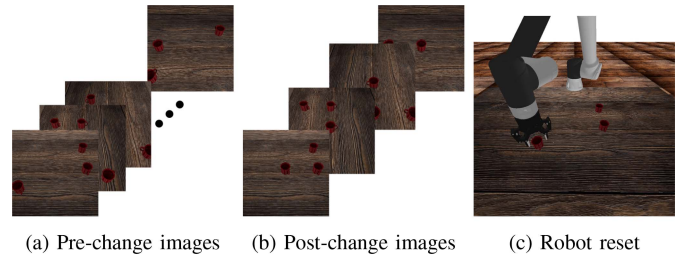


Fig. 8. 3DGS-CD enables robot workspace reset.

TABLE III
QUANTITATIVE EVALUATION OF 3DGS OR NeRF UPDATE METHODS ON THE SODA SCENE FROM NeRF-UPDATE DATASET [9]

Methods	Move-in		Move-out		time (s)
	PSNR	SSIM	PSNR	SSIM	
3DGS-update (Ours)	17.01	0.46	15.51	0.31	49
NeRF-update	17.39	0.50	14.61	0.30	168
InstantSplat	15.35	0.36	13.47	0.14	31

We report the average PSNR ($\uparrow$) and SSIM ($\uparrow$) for the move-in and move-out regions on evaluation images, and the 3DGS or NeRF update or InstantSplat [17] training time.

kinematics to compute the joint motions. For simplicity, we use the initial gripper orientation as the grasp orientation and use the object’s centroid as the grasp position.

Although our demo simplifies the grasp pose generation and collision avoidance process, the object change information our method provides can potentially enable more advanced motion planning algorithms.

C. Fast Sparse-View-Guided 3DGS Update

After training a 3DGS model for an initially static scene, if changes are made to the scene, our method enables the update of the initial 3DGS to reflect physical changes with the guidance of *sparse* post-change images.

Thanks to the explicit nature of the 3DGS, the Gaussians representing changed objects can be easily transformed to reflect object re-configurations. A more challenging task, however, is reconstructing the previously un-observed regions in the 3DGS model that are revealed by the object rearrangement, such as the part of the table that was previously occluded by the moved mustard bottle in Fig. 3(a). Instead of relying on in-painting-based solutions (e.g., GaussianEditor [28]) to infer these missing parts, we directly leverage supervision from post-change images to fill in the gaps.

In our method, we transform the in-object pre-optimized Gaussians according to the estimated object movements, and freeze all pre-trained Gaussian parameters. We then duplicate the Gaussians near the newly un-occluded regions and optimize only the duplicates to fill in the missing parts. Since these Gaussians are usually similar in position and color to the missing ones, they provide a good initialization for subsequent 3DGS optimization under sparse-view supervision. The Gaussians are optimized for 1000 step, during which we use the adaptive control strategy [8] to split, duplicate, reset and cull Gaussians every 100 steps.

We test our method on the Soda scene from the NeRF-Update dataset [9], as shown in Fig. 9. In this scene, 174 pre-change images are used to train a 3DGS and 4 post-change images serve as guidance for the update. We evaluate our method on reconstruction quality for the soda can’s move-in and move-out regions with PSNR and SSIM, and also on runtime performance,

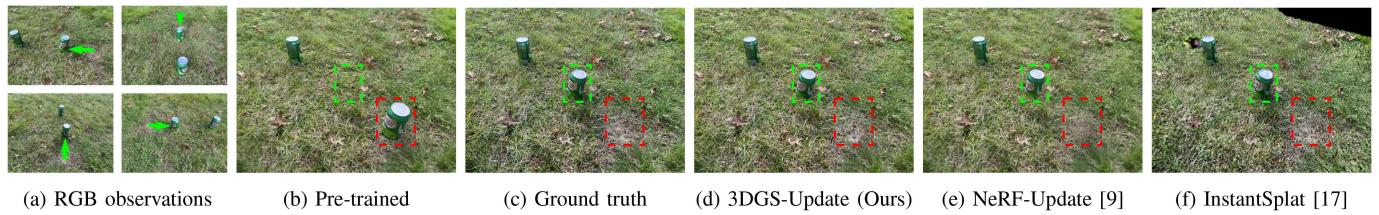


Fig. 9. 3DGS- or NeRF-update results: (a) Sparse RGB observations guiding NeRF or 3DGS update; (b) Evaluation-view renders by pre-trained 3DGS; (c) Ground truth evaluation images; (d-f) Evaluation-view renders by our method and baselines. The object move-in and move-out regions are highlighted with green and red boxes respectively.

as reported in Table III. Our method achieves comparable reconstruction quality and superior runtime performance compared to the NeRF-update [9] method. We also compare it with the SOTA sparse-view 3DGS training method, InstantSplat [8], which is only trained on the 4 post-change images. While InstantSplat is faster, it delivers lower reconstruction quality.

VII. LIMITATIONS

Non-rigid object changes: Our method represents object pose changes as 6DoF rigid transformations, which restricts its immediate application to non-rigid object changes. However, our pipeline is designed to be modular. Apart from the object pose estimation module, other components do not depend on the rigid object assumption. This allows for easy integration of a non-rigid pose estimation method to accommodate such changes.

Severe occlusions: Our method may fail if the changes are heavily occluded in all post-change views. This could result in incomplete object templates and cause the object 3D segmentation to fail. To mitigate this, we recommend capturing post-change images from angles that minimize occlusions, whenever possible.

VIII. CONCLUSION

We develop a novel 3DGS-based change detection method for identifying 3D object-level changes in complex real-world environments. Our approach significantly improves the accuracy and efficiency of radiance-field-based 3D change detection, enabling a wide range of real-world applications.

REFERENCES

- [1] M. Fehr et al., "TSDF-based change detection for consistent long-term dense reconstruction and dynamic object discovery," in *Proc. 2017 IEEE Int. Conf. Robot. Autom.*, 2017, pp. 5237–5244.
- [2] R. Finman, T. Whelan, M. Kaess, and J. J. Leonard, "Toward lifelong object segmentation from change detection in dense RGB-D maps," in *Proc. 2013 Eur. Conf. Mobile Robots*, 2013, pp. 178–185.
- [3] J. Fu, Y. Du, K. Singh, J. B. Tenenbaum, and J. J. Leonard, "Robust change detection based on neural descriptor fields," in *Proc. 2022 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 2817–2824.
- [4] T. Pollard and J. L. Mundy, "Change detection in a 3D world," in *Proc. 2007 IEEE Conf. Comput. Vis. Pattern Recognit.*, 2007, pp. 1–6.
- [5] K. Sakurada, T. Okatani, and K. Deguchi, "Detecting changes in 3D structure of a scene from multi-view images captured by a vehicle-mounted camera," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2013, pp. 137–144.
- [6] E. Palazzolo and C. Stachniss, "Fast image-based geometric change detection given a 3D model," in *Proc. 2018 IEEE Int. Conf. Robot. Autom.*, 2018, pp. 6308–6315.
- [7] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [8] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–14, 2023.
- [9] Z. Lu et al., "Fast sparse view guided NeRF update for object reconfigurations," 2024, *arXiv:2403.11024*.
- [10] R. Huang, B. Jiang, Q. Zhao, W. Wang, Y. Zhang, and Q. Guo, "C-NeRF: Representing scene changes as directional consistency difference-based NeRF," 2023, *arXiv:2312.02751*.
- [11] Y. Xiong et al., "Efficientsam: Leveraged masked image pretraining for efficient segment anything," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16111–16121.
- [12] P. F. Alcantarilla, S. Stent, G. Ros, R. Arroyo, and R. Gherardi, "Street-view change detection with deconvolutional networks," *Auton. Robots*, vol. 42, pp. 1301–1322, 2018.
- [13] R. Sachdeva and A. Zisserman, "The change you want to see (now in 3D)," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 2060–2069.
- [14] A. Kirillov et al., "Segment anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 4015–4026.
- [15] G. Wu et al., "4D Gaussian splatting for real-time dynamic scene rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 20310–20320.
- [16] P.-E. Sarlin, C. Cadena, R. Siegwart, and M. Dymczyk, "From coarse to fine: Robust hierarchical localization at large scale," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12716–12725.
- [17] Z. Fan et al., "Instantsplat: Unbounded sparse-view pose-free Gaussian splatting in 40 seconds," 2024, *arXiv:2403.20309*.
- [18] L. Ding, K. Zhu, D. Peng, H. Tang, K. Yang, and L. Bruzzone, "Adapting segment anything model for change detection in VHR remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5611711.
- [19] N. Otsu et al., "A threshold selection method from gray-level histograms," *Automatica*, vol. 11, no. 285–296, pp. 23–27, 1975.
- [20] D. DeTone, T. Malisiewicz, and A. Rabinovich, "Superpoint: Self-supervised interest point detection and description," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2018, pp. 224–236.
- [21] D. Wang et al., "MTP: Advancing remote sensing foundation model via multi-task pretraining," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 11632–11654, 2024.
- [22] V. Ye et al., "gsplat: An open-source library for Gaussian splatting," 2024, *arXiv:2409.06765*.
- [23] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "Lightglue: Local feature matching at light speed," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 17627–17638.
- [24] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer, "D-nerf: Neural radiance fields for dynamic scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 10318–10327.
- [25] M. Lebedev, Y. V. Vizilter, O. Vygolov, V. A. Knyaz, and A. Y. Rubis, "Change detection in remote sensing images using conditional adversarial networks," *Int. Arch. Photogrammetry, Remote Sens. Spatial Inf. Sci.*, vol. 42, pp. 565–571, 2018.
- [26] J. Cen et al., "Segment anything in 3D with radiance fields," 2023, *arXiv:2304.12308*.
- [27] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation in robotics, games and machine learning," 2016. [Online]. Available: <http://pybullet.org>
- [28] Y. Chen et al., "Gaussianeditor: Swift and controllable 3D editing with Gaussian splatting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 21476–21485.